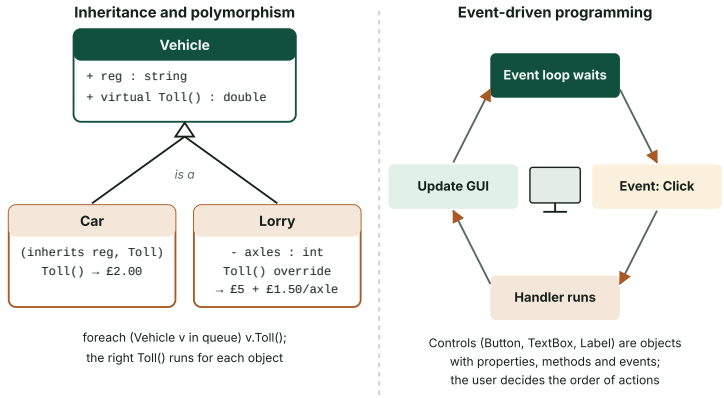
Inheritance, polymorphism, evaluating OOP and event-driven programming

AP25 FACT FILE

Specification: students should be able to

- Explain inheritance using the terms superclass (base class) and subclass (derived class), and represent a class hierarchy in a diagram
- Write C# subclasses that inherit from a base class, call the base constructor and add their own attributes and methods
- Explain polymorphism and implement it by overriding methods (virtual, abstract and override)
- Evaluate the object-oriented approach, discussing its advantages and disadvantages
- Describe event-driven programming and the use of events, event handlers and controls in a graphical user interface

Key words: inheritance, superclass, base class, subclass, derived class, class hierarchy, protected, base constructor, abstract class, polymorphism, virtual, override, overloading, code reuse, event-driven, event, event handler, control



Inheritance hierarchy with Car and Lorry deriving from Vehicle and Lorry overriding Toll, beside an event-driven cycle of event loop, event, handler and update

Building on classes: the big picture

Once a programmer can write classes and create objects, the next step is to make classes work together. Inheritance allows a new class to be built from an existing one, so a sports club system can define a general Member class once and then derive Junior and Adult classes that inherit everything a member has while adding their own details. Polymorphism builds on inheritance: the same method call, such as Fee(), can produce different results depending on which kind of object receives it, so a single loop can process a mixed list of members without checking each one's type. Together with encapsulation, these are the features that make object-oriented programming powerful for large systems. However, OOP is not automatically the best choice, so programmers must be able to evaluate it, weighing benefits such as realistic modelling, code reuse and reduced testing against extra complexity and a poor fit for some problems. Finally, this lesson looks at event-driven programming, in which the order of execution is decided by events such as button clicks. Graphical user interfaces are the most familiar example, and their buttons, text boxes and labels are themselves objects.

Inheritance

Inheritance is the mechanism by which a new, more specialised class takes on the attributes and methods of an existing, more general class. The general class is called the superclass, base class or parent class, and the specialised class is the subclass, derived class or child class. In C# the relationship is written with a colon, as in class Junior : Member. A subclass automatically has every attribute and method of its superclass and normally adds members of its own, so attributes accumulate as you move down a class hierarchy: if Member has two attributes and Junior adds one, a Junior object holds three. A subclass does not have to add anything, but usually does. Subclasses are independent of each other, so a method defined only in Junior cannot be used on an Adult object. Inheritance models an 'is a' relationship: a junior is a member, and a bus is a vehicle. If that sentence sounds wrong, inheritance is probably the wrong design. Hierarchies can have several levels, and a class hierarchy diagram shows the superclass at the top with lines and hollow arrowheads pointing up to it from each subclass.

Constructors, protected members and abstract classes

A subclass has its own constructor, but the superclass's attributes still need to be initialised. In C# the subclass constructor passes values up to the superclass constructor using : base(...), for example public Junior(int no, string n, string phone) : base(no, n), and then sets its own new attributes in its body. Private attributes of the superclass are inherited in the sense that every subclass object contains them, but code written in the subclass cannot refer to them directly; it must use the superclass's public get and set methods. Declaring a member protected, shown by # in a class diagram, makes it accessible inside the superclass and its subclasses while keeping it hidden from all other code. Sometimes a superclass describes an idea so general that no object should ever be created from it directly, such as Account in a credit union system where every account is in fact either a SavingsAccount or a LoanAccount. Declaring the class abstract prevents instantiation, so new Account(...) will not compile. An abstract class may contain abstract methods, which have a header but no body, and each non-abstract subclass is then forced to provide its own overriding version.

Polymorphism and overriding

Polymorphism, from the Greek for 'many forms', is the ability of objects of different classes to respond to the same method call in their own way. In C# it is achieved by overriding. The base class declares a method as virtual, meaning subclasses are allowed to replace it, or as abstract, meaning they must. A subclass then supplies a method with the same name, parameters and return type marked override. When a method is called through a reference of the base-class type, such as each item in a List<Vehicle>, the version that runs is chosen at run time from the actual class of the object, so a Car is charged the standard toll and a Lorry its axle-based toll. The calling code does not need an if statement to test each object's type, and a new subclass such as Coach can be added later without changing that loop. Overriding should not be confused with overloading, where several methods in the same class share a name but have different parameter lists. The compiler chooses the overloaded version from the arguments, at compile time, and no inheritance is involved.

Evaluating the object-oriented approach

OOP brings three big gains: realistic modelling, productivity and maintainability. Because classes correspond to real-world entities, the design is easier to discuss with users and maps naturally onto the problem. Inheritance and class libraries mean code is written once and reused, which improves productivity, and testing effort is reduced because a superclass that has already been tested does not need retesting; only the new or overridden methods in a subclass do. Encapsulation keeps each class's data safe from accidental change, and well-defined interfaces let a team develop and test classes separately and make maintenance easier. There are disadvantages. OOP suits some applications far better than others. It has been widely adopted for computer-aided design and manufacturing, engineering design, simulations, games and graphical interfaces, where objects are obvious, but programs that are very abstract or purely computational do not map neatly onto objects. For a small program or a straightforward data-processing job the extra design, code and memory overhead may bring little benefit. Deep hierarchies can become hard to understand, and a mistake in a base class spreads to every subclass. A good evaluation applies these points to the actual context and reaches a justified conclusion.

Event-driven programming and GUIs

In a sequential procedural program, the code decides the order in which steps happen. An event-driven program instead waits for events and responds to them, so the order of execution depends on what the user, the hardware or other software does. An event is an action or occurrence that the program can detect, such as a mouse click, a key press, a timer tick, a sensor reading or a message arriving. The program runs an event loop that listens for events and passes each one to the matching event handler, which is a subroutine attached to a particular control and event, such as btnCalculate_Click. Graphical user interfaces are the best-known example. Their controls, including buttons, text boxes, labels, check boxes, list boxes and timers, are objects with properties such as Text and Enabled, methods, and events they can raise, which shows how closely GUI programming and OOP are connected. The event-driven model suits interactive software because users choose what to do next, but programmers must make sure every handler works whatever order events occur in, for example by disabling a Save button until a file is open.

T Key terms from the mark schemes

Inheritance
The mechanism by which a new, more specialised class (subclass) takes on the attributes and methods of an existing, more general class (superclass) and can add or modify its own.

Derived class
A more specialised class that inherits from a base class.
Credited: Subclass / child class

Abstract class
A class that cannot be instantiated; objects must be created from its subclasses, which must implement any abstract methods it declares.

Overriding
Redefining an inherited method in a subclass with the same name and parameters, so the subclass's version runs for its objects.

Event-driven programming
A programming paradigm in which the flow of the program is determined by events such as user actions, sensor outputs or messages.

Control
An object on a graphical user interface, such as a button or text box, with properties, methods and events.
Credited: GUI component / widget

Base class
A general class whose attributes and methods are inherited by other classes.
Credited: Superclass / parent class

Protected
An access modifier that makes a member visible inside its own class and any subclasses but hidden from other code; shown as # in a class diagram.

Polymorphism
The ability of objects of different classes to respond to the same method call in different ways.
Credited: Many forms

Overloading
Defining several methods in the same class with the same name but different parameter lists.

Event handler
A subroutine that is run automatically when a particular event occurs on a control.

C C# example

Worked example: a GAA club membership system

```
Junior j = new Junior(101, "Orla", "07700 900123");
Console.WriteLine(j.GetName() + " " + j.GetMemberNo());   // Orla 101 (inherited)
Console.WriteLine(j.Fee());                                // 40 (overridden)
Console.WriteLine(j.Contact());                            // Orla, guardian 07700 900123
class Member
{
    private int memberNo;
    protected string name;
    public Member(int no, string n) { memberNo = no; name = n; }
    public int GetMemberNo() { return memberNo; }
    public string GetName() { return name; }
    public virtual double Fee() { return 120; }
}
class Junior : Member
{
    private string guardianPhone;
    public Junior(int no, string n, string phone) : base(no, n) { guardianPhone = phone; }
    public string Contact() { return name + ", guardian " + guardianPhone; }
    public override double Fee() { return 40; }
}
```

Exam focus

A complete definition of inheritance includes both ideas: a new, more specialised subclass takes on the attributes and methods (behaviours) of an existing superclass, and it can add its own or modify inherited ones. Use the terms superclass or base class and subclass or derived class correctly, and be ready to count how many attributes a subclass object has by adding inherited and new ones. For polymorphism, say that the same method call produces different behaviour for objects of different classes and support it with an example from the question, naming the overridden method. 'Evaluate' and 'discuss' questions on OOP are usually worth six marks or more and are marked in levels, so give developed points on both sides, such as more accurate modelling, reuse through inheritance and reduced testing, against unsuitability for abstract problems, extra complexity and overheads, then reach a conclusion that fits the scenario. Lists of single words earn little. When writing a subclass in C#, show the colon, call : base(...) in the constructor and use override where a method is replaced. For event-driven questions, use the terms event, event handler and control accurately and describe the sequence from event to output.

? Check yourself

1. A Fermanagh rugby club's system has a Member class and a new Junior class built from it. Explain what is meant by inheritance, using this example. [2]
2. A council's pitch-booking system has a superclass Booking with the attributes bookingRef, bookingDate and customerName. The subclass PitchBooking adds the attributes pitchNumber and floodlights. State how many attributes a PitchBooking object has and explain why. [2]
3. Explain what is meant by polymorphism, using an example. [3]

Answers: 1. A new, more specialised class (subclass / derived class, here Junior) takes on / inherits the attributes and methods of an existing, more general class (superclass / base class, here Member) (1) / and can add its own attributes and methods, e.g. a guardian's phone number, or modify inherited ones (1) 2. 5 (1) / it inherits all 3 attributes of Booking and adds 2 of its own (1) 3. The same method call / message (1) / produces different behaviour depending on the class of the object it is called on (1) / example, e.g. Fee() returns £120 for an Adult object but £40 for a Junior object because Junior overrides it (1)